SMART CAMPUS EVENT MANAGEMENT SYSTEM

School of Computing

Bachelor of Technology
in
Computer Science & Engineering

By

M.V.Yaswanth Kumar(VTU26623)

10211CS224
Full Stack Application Development
Slot :E1
Bucket-4

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ”SMART CAMPUS EVENT MANAGEMENT SYSTEM ” by M.V.Yaswanth Kumar (VTU26623) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.Hafizur Rahman

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr.Sumithra.M

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and ref- erenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke pe- nal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(M.V.Yaswanth)

Date: /05/2026

ABSTRACT

The Smart Campus Event Management Application is a full stack web-based system designed to streamline the organization, registration, and management of campus events. Traditional event handling methods often rely on manual processes, leading to inefficiencies, data loss, and poor user experience.

The application enables users to browse events, select ticket types, and complete registrations through an interactive and user-friendly interface. It incorporates dynamic frontend technologies for seamless navigation and backend services for secure data handling and storage. The system ensures accurate record keeping by storing participant details, ticket information, and booking confirmations in a structured database.It also enhances user engagement by offering instant confirmations and a structured booking experience. Overall, the project demonstrates how modern web technologies can be leveraged to build scalable, efficient, and user-centric solutions for campus event management.

Keywords: Event Management, Full Stack Development, Web Application, Ticket Booking System, Database Management, Campus Automation.

LIST OF FIGURES

1.1 LOGIN PAGE 4

1.2 ADMIN OTP VERIFICATION PAGE 4

1.3 ADMIN LOGIN PAGE 5

1.4 EVENTS PAGE 5

1.5 USER REGISTRATION PAGE 6

3.1 Booking Records Stored in MySQL Database..... 12

4.1 Proposed System Architecture of Smart Campus Event Management Application 14

LIST OF TABLES

5.1 Comparison of Existing Systems with Proposed System 19

8.1 SDG Mapping for the Project..... 25

8.2 Mapping of Project to Complex Engineering Activities (CEA)..... 26

8.3 Mapping of Project to Complex Engineering Problems (CEP)..... 27

LIST OF ACRONYMS AND ABBREVIATIONS

CI	Continuous Integration
ORM	Object Relational Mapping
UI	User Interface
UX	User Experience
API	Application Programming Interface
DBMS	Database Management System
SQL	Structured Query Language
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
HTTP	HyperText Transfer Protocol
CRUD	Create, Read, Update, Delete

TABLE OF CONTENTS

Page.No

ABSTRACT **iii**

LIST OF FIGURES **iv**

LIST OF TABLES **v**

LIST OF ACRONYMS AND ABBREVIATIONS **vi**

1 INTRODUCTION **1**

1.1 Introduction 1

1.2 Aim of the Project 2

1.3 Scope of the Project 2

1.4 Framework 3

2 SURVEY OF SIMILAR APPLICATION IN MARKET **7**

3 FRAMEWORK DESCRIPTION **9**

3.1 Frontend Implementation 9

3.2 Backend Implementation 10

3.3 Database Implementation 11

4 METHODOLOGIES 13

4.1 Project Architecture 13

4.2 DataFlow Diagram 14

4.3 Module 1: User Interface Module 15

4.4 Module 2: API and Backend Module 15

4.5 Module 3: Database Module 16

5 RESULTS AND DISCUSSIONS 18

6 CONCLUSION AND FUTURE ENHANCEMENTS 20

6.1 Conclusion 20

6.2 Future Enhancements 21

7 SOURCE CODE 22

8 ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG 25

8.1 SDG Mapping 25

8.2 Mapping of the Project to Complex Engineering Activities (CEA) 26

8.3 Mapping of Project to Complex Engineering Problems (CEP)..... 27

REFERENCES **28**

Chapter 1

INTRODUCTION

1.1 Introduction

Managing campus events through manual or semi-digital methods often leads to inefficiencies, lack of coordination, and poor user experience. To overcome these challenges, a Smart Campus Event Management Application is developed as a full stack web-based system that centralizes event management and ticket booking processes.

The application enables users to explore events, select ticket types, and register seamlessly through an interactive interface. It also provides organizers with a structured platform to manage event data, participant records, and bookings in real time. By integrating frontend and backend technologies, the system ensures efficient data flow, improved accessibility, and enhanced user engagement, making the entire event management process faster, reliable, and scalable.

1.2 Aim of the Project

The aim of this project is to design and develop a full stack web application for efficient management of campus events and ticket bookings. The system focuses on simplifying event registration, improving user interaction, and ensuring accurate data handling through a centralized platform. It also aims to reduce manual effort, enhance accessibility, and provide a seamless experience for both users and event organizers.

1.3 Scope of the Project

The scope of this project includes the development of a web-based application for managing campus events such as seminars, workshops, hackathons, and technical fests. The system allows users to view event details, select ticket types, and complete registrations, while enabling organizers to store and manage booking data efficiently.

The application emphasizes a responsive user interface, real-time interaction, and reliable data storage. Future enhancements may include integration of online payment systems, QR code-based ticket verification, email notifications, and analytics dashboards for monitoring event performance and user engagement.

1.4 Framework

The proposed Smart Campus Event Management Application is developed using a full stack architecture combining frontend and back- end technologies. The frontend is built using HTML, CSS, and JavaScript to create a responsive and user-friendly interface for event browsing and ticket booking.

The backend is developed using Node.js with Express.js, which handles server-side logic, API creation, and data processing. MySQL is used as the database management system to store and manage user details, event information, and booking records efficiently.

This architecture ensures smooth communication between the client and server, enabling real-time data handling, scalability, and efficient performance. The modular design allows easy expansion and integration of additional features, making the system adaptable for future improvements.

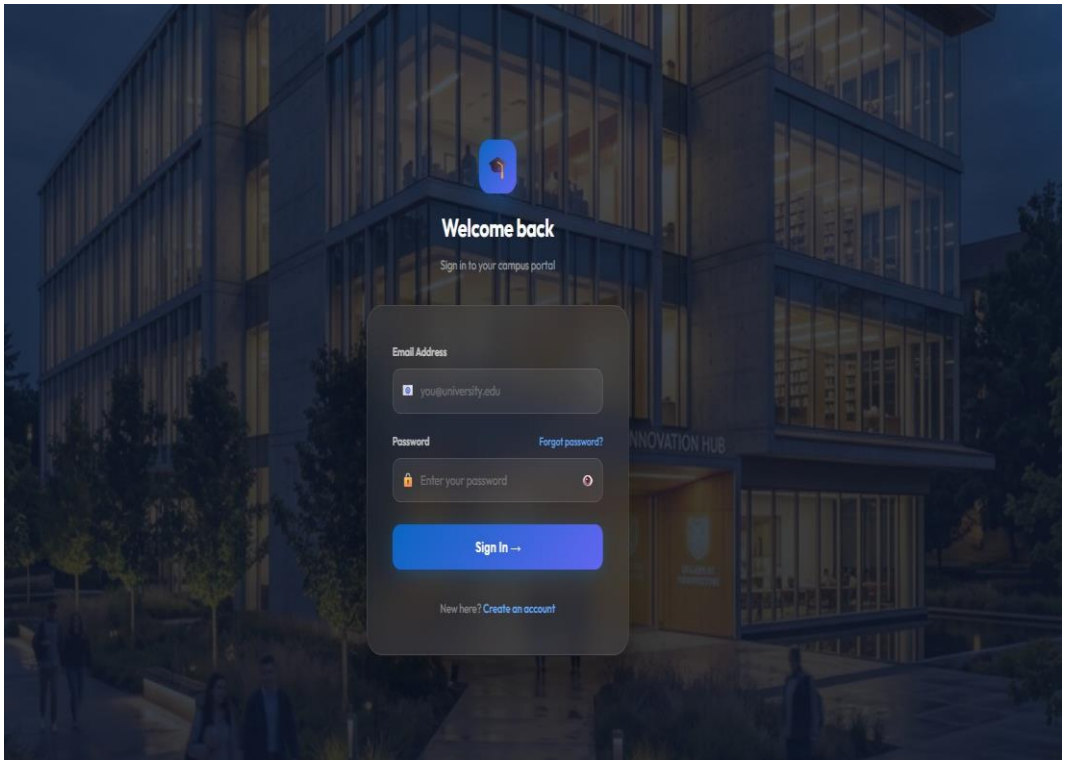

Figure 1.1: LOGIN PAGE

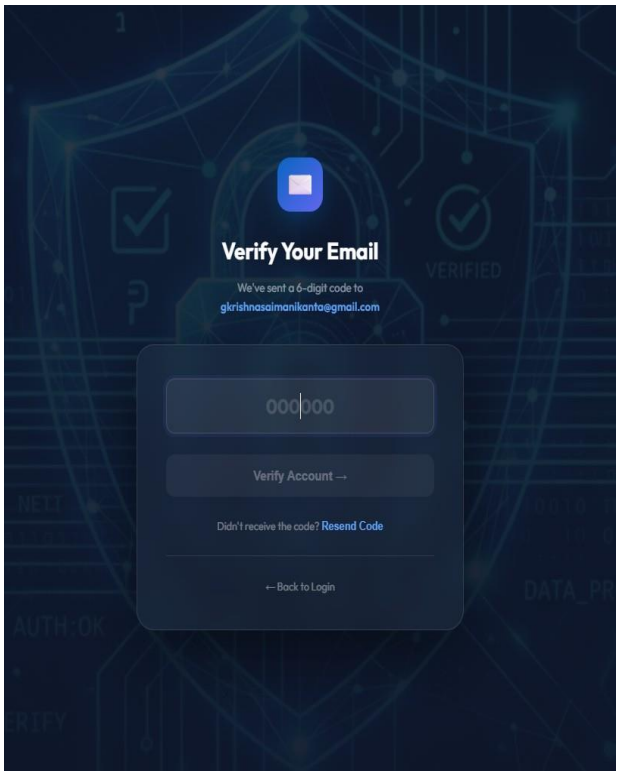

Figure 1.2: ADMIN OTP VERIFICATION PAGE

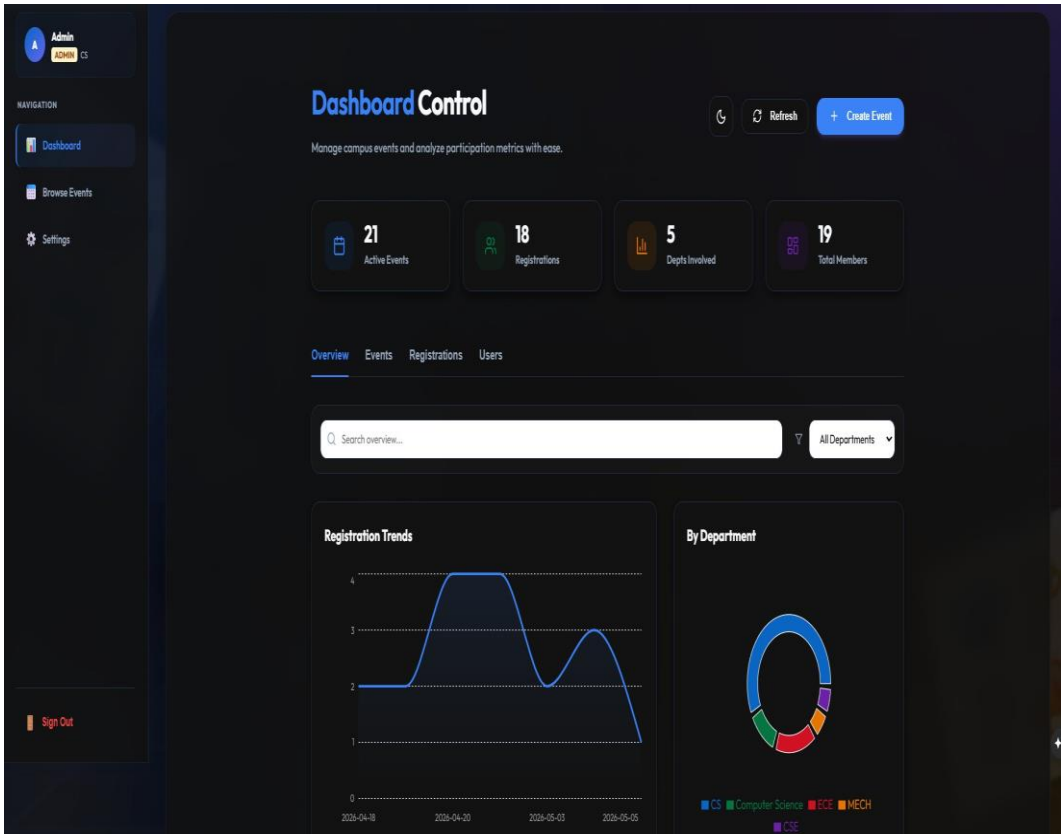

Figure 1.3: ADMIN LOGIN PAGE

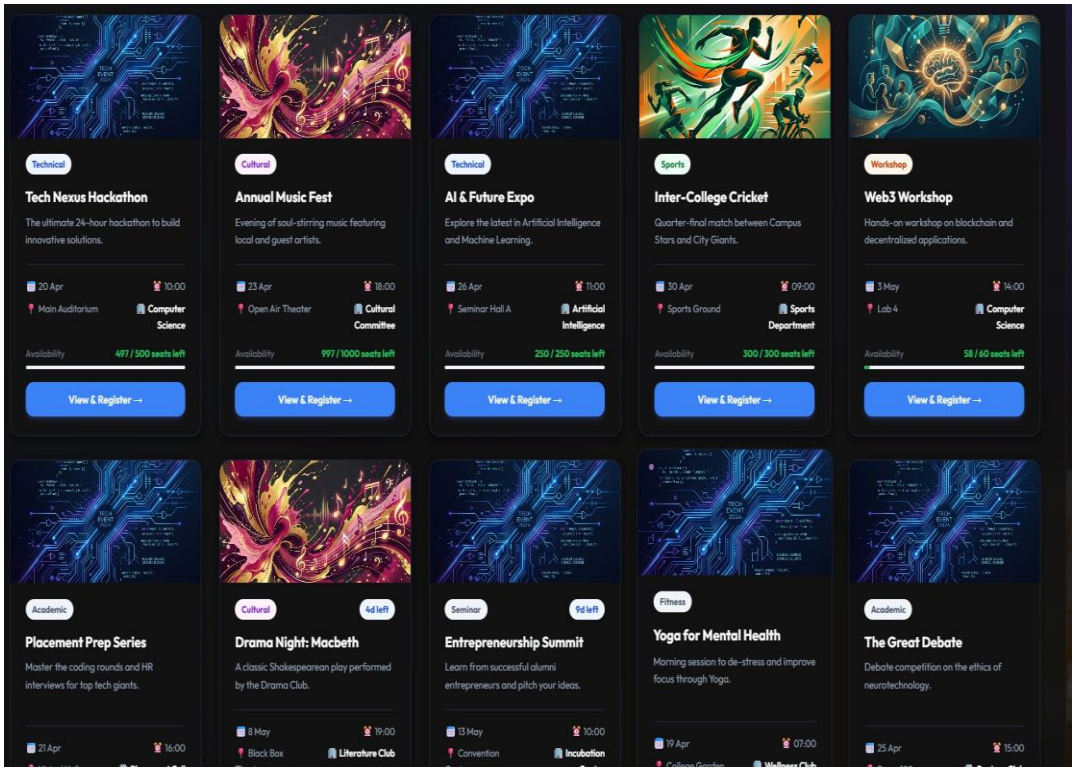

Figure 1.4: EVENTS PAGE

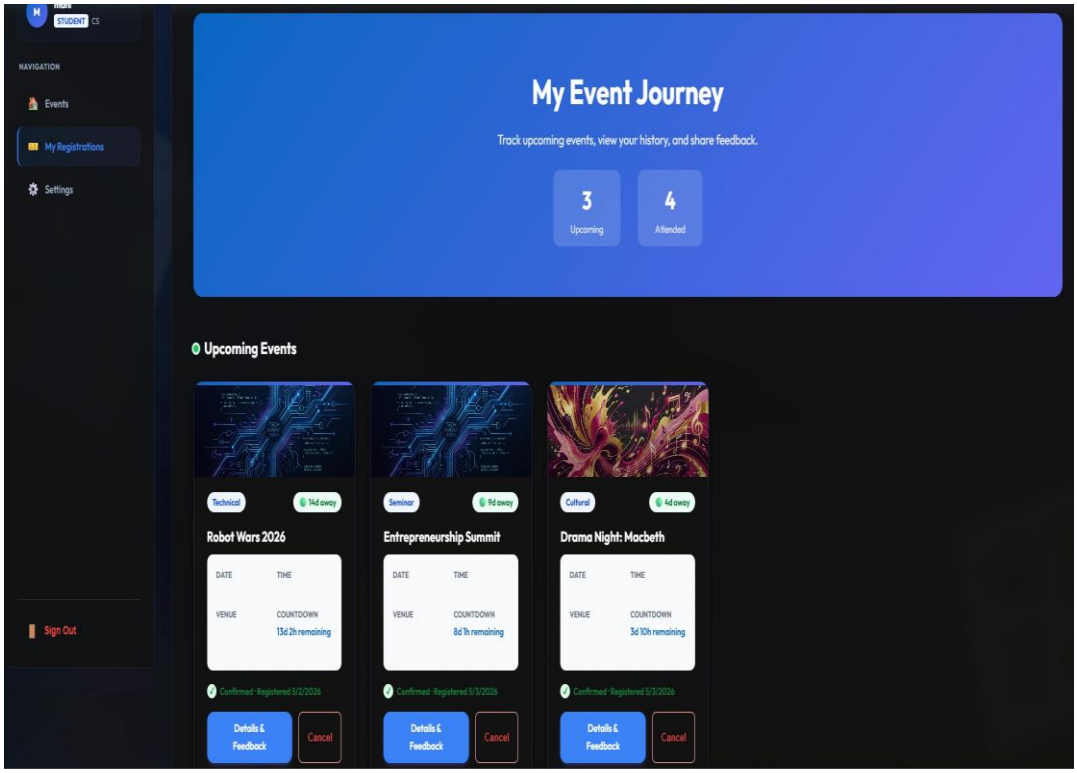

Figure 1.5: USER REGISTRATION PAGE

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

In the current market, several online event management and ticket booking platforms have been developed to simplify event discovery, registration, and reservation processes. One of the most widely used platforms in India is BookMyShow, which provides services for booking tickets for movies, concerts, sports events, and live shows. It offers features such as event listings, seat selection, secure payment integration, and a user-friendly interface, making it highly popular among users [ref1].

Similarly, global platforms like Ticketmaster and StubHub provide large-scale ticketing solutions with advanced features such as event promotion, dynamic pricing, secure transactions, and customer support. These systems are designed to handle millions of users and large-scale events efficiently, ensuring reliability and scalability [ref2].

Most modern event management applications share common functionalities including user registration, event browsing, ticket booking, real-time availability updates, and booking history tracking. Advanced platforms also include features such as personalized recommendations, analytics dashboards for organizers, QR-based ticket verification, and fraud detection mechanisms to enhance security and user experience [ref3].

Despite their advantages, existing systems often have certain limitations such as high service charges, complex user interfaces, dependency on online payments, and system overload during peak booking periods. Additionally, these platforms are generally designed for large-scale commercial events and may not be suitable for smaller environments like campus or departmental events.

This survey indicates that while current applications are powerful and feature-rich, there is a need for a simplified, cost-effective, and customizable solution tailored for academic institutions. The proposed Smart Campus Event Management Application addresses this gap by providing a lightweight, user-friendly, and efficient system specifically designed for managing campus events and registrations.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend development environment is set up using HTML, CSS, and JavaScript for building a responsive and interactive user interface. The project is developed using Visual Studio Code as the code editor. Node.js is used to support package management if required, and testing is performed using modern web browsers such as Google Chrome. The frontend communicates with the backend using HTTP requests to handle booking operations and data exchange.

3.1.2 Structure of the Project

The frontend follows a structured and modular design for better maintainability. The application consists of multiple sections such as home page, event listing page, and ticket booking page. JavaScript functions handle dynamic behaviors like ticket selection, form valida-

tion, and navigation between pages. CSS is used for styling and layout design to ensure a clean and responsive interface. The code is organized into sections for UI components, styling, and scripts to maintain clarity.

3.1.3 Execution Procedure

To execute the frontend, the project files are opened in a web browser. The user can navigate through different sections such as viewing events and booking tickets. When a user submits the booking form, the frontend sends data to the backend server through API calls. The response from the backend is then displayed to the user in the form of booking confirmation.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend environment is configured using Node.js and Express.js to handle server-side logic and API requests. MySQL is used as the database system for storing booking information. The server is developed and tested using Visual Studio Code, and the API is accessed locally through a server running on a specified port (e.g., localhost:5000).

3.2.2 Structure of the Project

The backend follows an API-based architecture. It includes end-

points such as /book for handling ticket bookings and /bookings for retrieving stored data. The server processes incoming requests, validates user input, and interacts with the MySQL database to store booking records. The structure includes modules for server configuration, database connection, and API handling, ensuring separation of concerns.

3.2.3 Execution Procedure

To run the backend, the MySQL database is first started and connected. The Node.js server is then executed using the command `node server.js`. Once the server is running, it listens for incoming requests from the frontend. When a booking request is received, the server processes the data and stores it in the database. The stored data can be verified using SQL queries such as `SELECT * FROM bookings;`.

3.3 Database Implementation

3.3.1 Database Configuration

The database is configured using MySQL to store and manage booking data. A database named `techfest` is created and selected us- ing SQL commands. The configuration is done through the MySQL command line interface, ensuring proper setup for data storage and

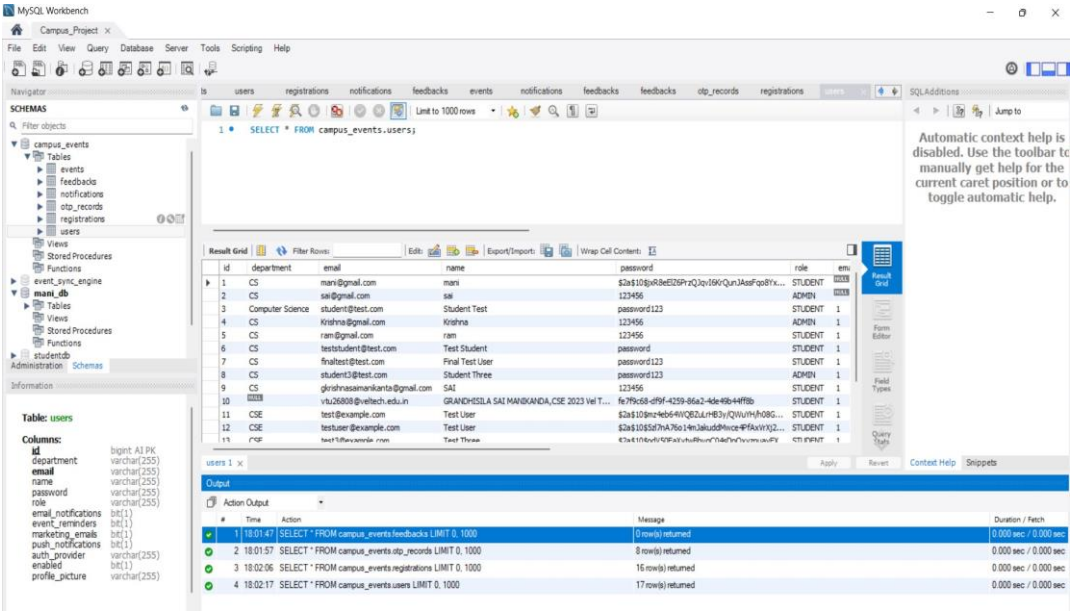

Figure 3.1: Booking Records Stored in MySQL Database

retrieval.

3.3.2 Schema Structure

The database schema is designed to handle booking information efficiently. It includes structured fields such as name, email, phone number, department, ticket type, event name, quantity, amount, and booking ID. These fields ensure accurate storage and retrieval of user booking details.

3.3.3 Tables created

The main table created in the database is the `bookings` table. It stores all booking-related information including user details, selected ticket type, event name, number of tickets, total amount, and unique booking ID. This table enables efficient tracking and management of event registrations within the system.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project follows a layered architecture consisting of presentation, application, and database layers. The presentation layer includes the frontend developed using HTML, CSS, and JavaScript, which provides an interactive interface for users to explore events and book tickets. The application layer is powered by a Node.js and Express backend that handles API requests, processes booking logic, and ensures smooth communication between the frontend and database. The database layer uses MySQL to store and manage event and booking records efficiently.

The frontend communicates with the backend through RESTful APIs, while the backend interacts with the database to perform data storage and retrieval operations. This separation of concerns improves scal- ability, maintainability, and performance of the system. Future en-

hancements such as authentication, payment gateways, and cloud deployment can be integrated seamlessly into this architecture.

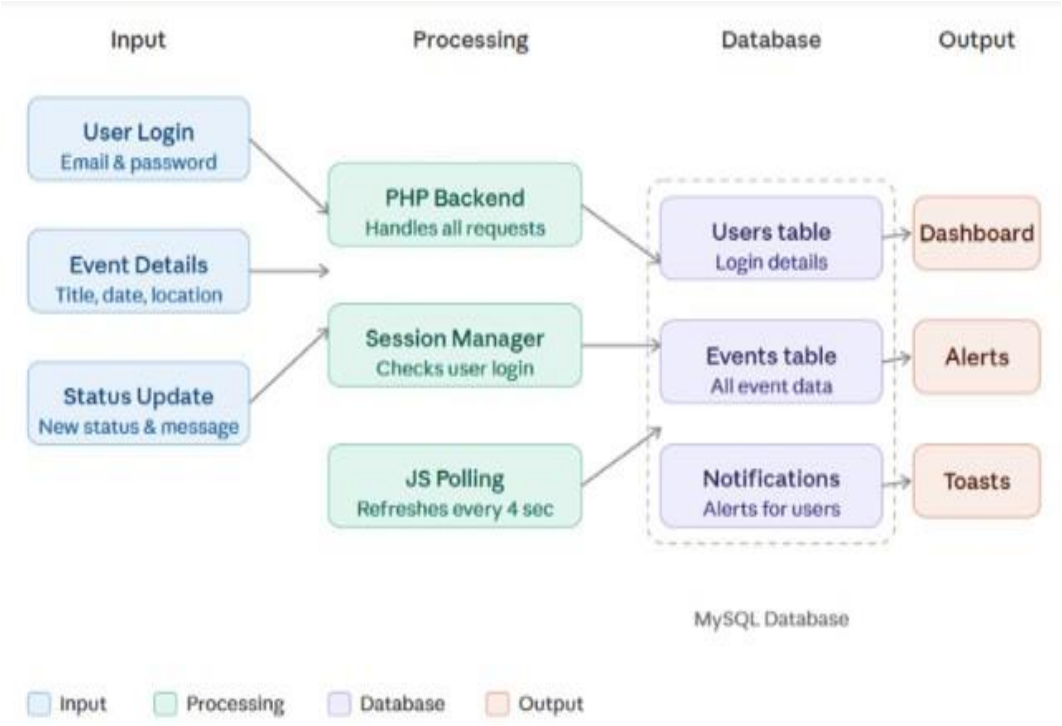

Figure 4.1: Proposed System Architecture of Smart Campus Event Management Application

4.2 DataFlow Diagram

The data flow begins when the user accesses the web application and views the list of available events. The frontend sends a request to the backend API to fetch event data. When a user selects an event and submits the booking form, the entered details are validated on the client side and sent to the backend server.

The backend processes the request, performs validation, and stores

the booking information in the MySQL database. It also updates ticket availability if required. After successful storage, a confirmation response is sent back to the frontend, which displays booking confirmation details to the user.

4.3 Module 1: User Interface Module

This module is responsible for the design and interaction of the application. It includes event listings, filtering options, ticket selection, and the booking form. The module ensures a clean and user-friendly experience using structured layouts, responsive design, and interactive elements.

4.4 Module 2: API and Backend Module

This module manages communication between the frontend and database. It includes API endpoints such as `/book` for handling ticket bookings and `/bookings` for retrieving booking records. The backend validates user inputs, processes business logic, and ensures reliable data transfer.

4.5 Module 3: Database Module

This module handles data storage and management. It uses MySQL to maintain structured tables such as bookings, which store user details, ticket type, selected event, quantity, and total amount. It ensures efficient data retrieval and consistency across the system.

Advantages of this Project

This project provides a streamlined and digital approach to managing campus event registrations and ticket bookings.

a) User-Friendly Interface: The system offers an intuitive and responsive interface, making it easy for users to browse events and complete bookings quickly.

b) Real-Time Data Handling: The application dynamically updates booking information, ensuring accurate and up-to-date ticket availability.

c) Centralized Data Management: All booking and event details are stored in a structured database, simplifying data access and management.

Disadvantages

The system has certain limitations that can be addressed in future improvements. It currently depends on a local server setup, limiting accessibility beyond a specific environment. Advanced features such

as secure authentication, online payment integration, and scalability for large-scale events are not yet implemented. Additionally, perfor- mance optimization may be required for handling high traffic scenar- ios.

Chapter 5

RESULTS AND DISCUSSIONS

The Smart Campus Event Management Application was successfully designed and implemented to streamline the process of event registration and ticket booking within an academic environment. The system integrates a responsive frontend interface with a Node.js backend and MySQL database, ensuring efficient data handling and smooth user interaction.

The application allows users to browse events, select ticket types, and complete registrations seamlessly. The booking details are stored in the database and retrieved in real-time, demonstrating the reliabil- ity of the system. The confirmation module generates an e-ticket with booking details, improving user experience and reducing manual ef- fort.

Testing results indicate that the system performs efficiently for small to medium-scale usage, with accurate data storage and retrieval. The frontend interface is responsive and user-friendly, while the backend

ensures proper validation and processing of booking data. Screenshots of the application interface, booking form, and database records are included below for reference.

Table 5.1: Comparison of Existing Systems with Proposed System

Applications	**Key Features / Limitations**
BookMyShow	High service charges, overkill for small campus events
Ticketmaster	Complex system, not suitable for internal academic use
StubHub	Focused on ticket resale, not event management
Traditional Manual System	No automation, high error rate, no tracking
Proposed System	Event-specific booking, real-time DB storage, simple UI, no extra cost

The results demonstrate that the proposed system effectively reduces manual work, improves booking accuracy, and provides a structured approach to event management. Compared to existing large- scale platforms, this system is optimized for academic institutions, making it simple, efficient, and easy to deploy. Future improvements can further enhance performance, scalability, and feature integration such as online payments and authentication.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event Management Application was successfully designed and implemented to digitize and simplify event registration and ticket booking within an academic environment. The system integrates a responsive frontend built using HTML, CSS, and JavaScript with a Node.js and Express backend, along with a MySQL database for efficient data storage.

The application enables users to browse events, select ticket types, and complete bookings through an intuitive interface. It ensures proper validation of user inputs, accurate calculation of ticket amounts, and reliable storage of booking details including event selection. The seamless communication between frontend and backend through API requests ensures real-time data handling and smooth user experience.

Compared to existing large-scale platforms, the proposed system is lightweight, cost-effective, and specifically tailored for campus-level events. It reduces manual effort, minimizes errors, and provides a structured approach to managing event registrations efficiently.

6.2 Future Enhancements

The current system lays a strong foundation but can be significantly enhanced to create a more powerful and scalable platform. Future improvements include:

- Implementing user authentication and login system for secure and personalized access
- Integrating online payment gateways for real-time ticket booking and transactions
- Adding email and SMS notifications for instant booking confirmation and reminders
- Development of an admin dashboard for managing events, tracking bookings, and analytics
- Implementing QR code-based e-ticket verification for faster check-in
- Deploying the application on cloud platforms for high availability and scalability

Chapter 7

SOURCE CODE

```
const express = require("express");
const mysql = require("mysql2");
const app = express();

app.use(express.json());

const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "techfest"
});

app.post("/book", (req, res) => {
  const { name, email, phone, ticketType, quantity } = req.body;
  const amount = ticketType * quantity;
  const bookingId = "EVT" + Math.floor(Math.random()*10000);

  const sql = "INSERT■INTO■bookings■(name,email,phone,ticketType,quantity)";

  db.query(sql, [name,email,phone,ticketType,quantity,amount,bookingId],
    (err,result)=>{
      if(err) throw err;
      res.json({bookingId});
    });
});
app.listen(5000, ()=> console.log("Server■running"));

language=HTML]
<!DOCTYPE html>
<html>
<head>
<title>Event Booking</title>
</head>
```

```
<body>

<h2>Book Your Event</h2>

<form id="bookingForm">
  Name: <input type="text" id="name"><br>
  Email: <input type="email" id="email"><br>
  Phone: <input type="text" id="phone"><br>
  Ticket Type:
  <select id="ticketType">
    <option value="100">Standard - 100</option>
    <option value="200">VIP - 200</option>
  </select><br>
  Quantity: <input type="number" id="quantity"><br>
  <button type="submit">Book Now</button>
</form>

<script>
document.getElementById("bookingForm").addEventListener("submit", async function(e) {
  e.preventDefault();

  const data = {
    name: document.getElementById("name").value,
    email: document.getElementById("email").value,
    phone: document.getElementById("phone").value,
    ticketType: document.getElementById("ticketType").value,
    quantity: document.getElementById("quantity").value
  };

  const res = await fetch("http://localhost:5000/book", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify(data)
  });

  const result = await res.json();
  alert("Booking Successful! ID: " + result.bookingId);
});
</script>

</body>
</html>


[language=SQL]
CREATE TABLE bookings (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100),
```

```
  phone VARCHAR(15),
  ticketType VARCHAR(20),
  quantity INT,
  amount INT,
  bookingId VARCHAR(20)
);
```

Chapter 8

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

8.1 SDG Mapping

Table 8.1: SDG Mapping for the Project

SDG No.	SDG Title	Relevance to the Project
SDG 9	Industry, Innovation and Infrastructure	Promotes digital infrastructure and automation in campus systems
SDG 11	Sustainable Cities and Communities	Supports smart campus ecosystem through efficient event management
SDG 12	Responsible Consumption and Production	Reduces paper usage by enabling digital ticket booking
SDG 16	Peace, Justice and Strong Institutions	Ensures secure data handling and reliable system operations

8.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 8.2: Mapping of Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of diverse technologies	HTML, CSS, JavaScript, Node.js, Express, and MySQL used together
EA2	Level of Interactions	Complex module interactions	API-based communication between frontend, backend, and database
EA3	Innovation	Modern technologies	Real-time booking system with dynamic UI and database updates
EA4	Societal Impact	Impact on users	Improves efficiency of campus event management and reduces manual work
EA5	Familiarity	Beyond standard practices	Full stack system integrating multiple layers and real-time processing

8.3 Mapping of Project to Complex Engineering Problems (CEP)

Table 8.3: Mapping of Project to Complex Engineering Problems (CEP)

Level	Attribute	Characteristics	Justification
WP1	Depth of Knowledge	Requires engineering knowledge across multiple domains	Full stack development involving frontend, backend, and database technologies
WP2	Conflicting Requirements	Technical and usability trade-offs	Balancing performance, user experience, and data consistency
WP3	Depth of Analysis	Analytical and design thinking	Designing APIs, database schema, and system workflows
WP4	Familiarity	Partially new problem domain	Customized solution for campus-specific event management
WP5	Applicable Codes	Limited standardization	Custom booking logic without strict external standards
WP6	Stakeholder Needs	Multiple stakeholders involved	Students, organizers, and administrators interacting with system
WP7	Interdependence	Integration across multiple layers	Strong dependency between frontend, backend, and database

REFERENCES

[1] Meta Platforms Inc. (2024). *React: A JavaScript Library for Building User Interfaces*, React Official Documentation, 1(1), pp. 1–10. https://react.dev/

[2] OpenJS Foundation (2024). *Node.js Runtime Environment Documentation*, Node.js Official Docs, 2(1), pp. 1–12. https://nodejs.org/en/docs

[3] Oracle Corporation (2024). *MySQL Relational Database Management System*, MySQL Official Documentation, 3(2), pp. 1–15. https://dev.mysql.com/doc/

[4] BookMyShow Team (2023). *Online Ticket Booking System Architecture and Implementation*, BookMyShow Technical Report, 45–52. https://tech.bookmyshow.com/

[5] Eventbrite Inc. (2023). *Event Management and Ticketing Platform System Design*, Eventbrite Engineering Journal, 4(3), pp. 30–38. https://www.eventbrite.com/engineering/